

Throw Control — Unity Asset by makaka.org

Disclosure: This website may contain affiliate links, which means I may receive a commission (as an associate of Unity Asset Store and other 3rd parties) if you click a link and purchase something that I have recommended. While clicking these links won't cost you any money, they will help me fund my dev projects while recommending great assets!

[Throw Control \(Unity Asset\)](#) — highly customizable Advanced 3D Throwing System for Unity. You can throw any object forward or in any direction with custom force, a center of mass, and 80+ other throwing parameters. 3 Modes. 4 Demos. 11 Throwing Objects.

Note: The asset is downloaded into your project as a Template (Complete Project).

[Read the "Getting Started" Tutorial](#) to Import it correctly.

Check out the [Support section](#) if you need help.

Contents [\[hide\]](#)

1. Features of Throw Control
2. Package Contains
3. Package is a Part of Unity Assets
4. Use Cases
5. Throwing System: 80+ Customizable Parameters
6. Tutorial
 - 6.1. Getting Started with Throw Control
 - 6.3. Creating your own Throwing Object Prefab
 - 6.4. How to Keep Game Objects in Scene after the Throwing?
7. Throw and Score (Scene)
8. Testing
9. Support

Features of Throw Control

All modules are designed independently to keep the Unity Asset extendable & easy to understand:

- ★ Cross-platform (Mobile, Desktop) Throwing with 3 Modes:
 - ★ **"Click Or Tap"** (Easy),
 - ★ **"Flick"** (or Swipe: Hard),
 - ★ **"Press Key"** (Easy).
- ★ Device Orientations: Portrait, Landscape.
- ★ 80+ Customizable Parameters — sensitivity, force, torque, delays, position, rotation, fading, etc.:
 - ★ for all Throwing Objects at once;
 - ★ for each Throwing Object individually.
- ★ 4 Demos: Mobile, Desktop.
- ★ 7 Events.
- ★ 11 customized Throwing Objects:
 - ★ Weapons: Axe, Sword, Broken Sword;
 - ★ Balls: Basketball, Brick Balls (Grey, Red), Football;
 - ★ Miscellaneous: Coin, Shovel, Horseshoe, Tree/Branch.
- ★ Easy implementation of your own Throwing Objects.
- ★ Dynamic Sound System — play Sounds based on speed, pitch, and volume factors of Throwing Objects upon:
 - ★ Throw,
 - ★ Custom Game Logic (e.g., for Customizing Collisions with Different Surfaces: floor, wall...).
- ★ Dynamic Material System:
 - ★ Fading In & Fading Out: Dissolving/Appearing VFX,
 - ★ Material Changing in Runtime for Any Custom Game Logic (e.g., **fail**, **win**),
 - ★ Preventing Redundant Memory Allocations.
- ★ Layer Changing: to avoid unwanted and mutual collisions.
- ★ Object Pool for Throwing Objects to manage the memory.

Package Contains

- ★ Cross-Platform **Throw and Score** Scene (Game Template) with [Sensor Camera](#) (included):
 - ★ Toss Diligently and Try to Get the Best Score by Hitting the Target Containers without touching the Ground (Mode: **"Click Or Tap"**).
- ★ Desktop FPS Demo Scene with Controls: Walk, Run, Jump, Throw (Mode: **"Click"**).
- ★ 2 Demo Scenes with Static Camera:
 - ★ Cross-Platform (Mode: **"Flick"**).
 - ★ Desktop (Mode: **"Press Key"**). It also works on Mobiles with a separate Throwing Mode (**"Click Or Tap"**) that you can indicate in the Editor.
- ★ Menu Scene.
- ★ Loading Screen to switch scenes seamlessly.

[Check the Map of Unity Assets](#) to choose the product that best suits your needs.

Package is a Part of Unity Assets

[Football \(docs\)](#).

Football – Unity Asset 



[Basketball Game \(docs\)](#).

Basketball Game 3D 🏀 Unity Asset 🏀 Throw Ball o...



Use Cases

- ★ Games in the style of:
 - ★ Pokemon GO, Can Knockdown, Paper Toss Boss, Paper Bin AR;
- ★ AR Unity Assets by Makaka Games;
- ★ Throw Flow Game;
- ★ Sport Games: Basketball, Football.
- ★ FPS, VR Games with Throwing Objects;
- ★ Survival Games;
- ★ Any of your Bold Fantasies.

Throw Flow 🍊 Level 3 🍊 Developer Prototype for C...



Throw 3D Object in Unity ⭐ Knives in Desert ⭐ Thr...



Throwing System: 80+ Customizable Parameters

“

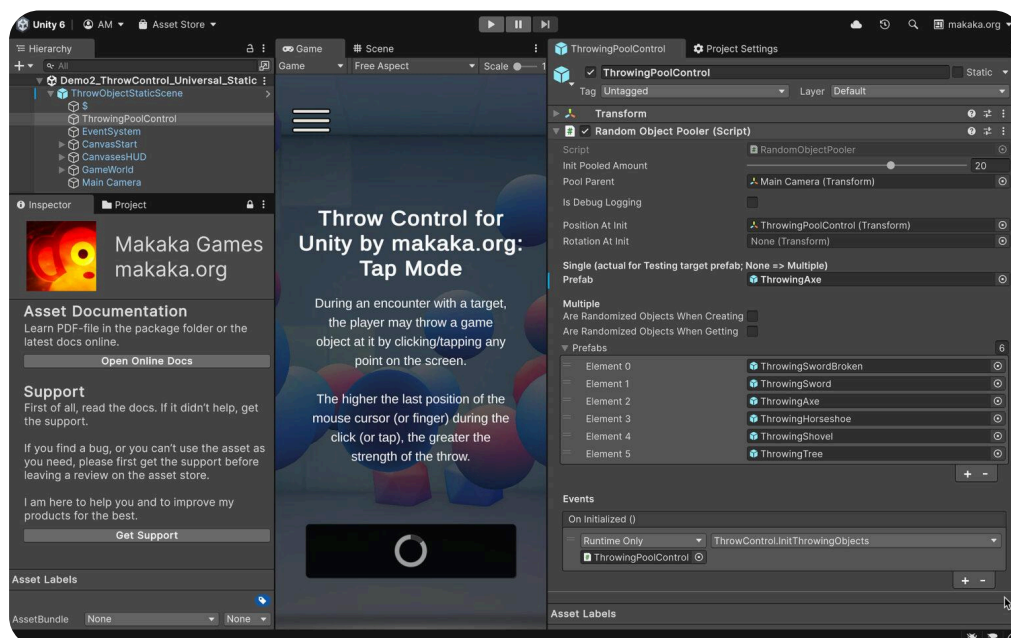
Throw Control (Unity Asset) implements a **Throwing System** consisting of 4 Control Scripts that are fully customized in Demo Scenes.

Scripts of Throwing System:

- ★ *RandomObjectPooler.cs* and *ThrowControl.cs* operate All Throwing Objects at once and exist on the Scene initially on **ThrowingPoolControl** Game Object.
- ★ *ThrowingObject.cs* and *MaterialControl.cs* are attached to Each Throwing Object (Each Prefab) and control it.

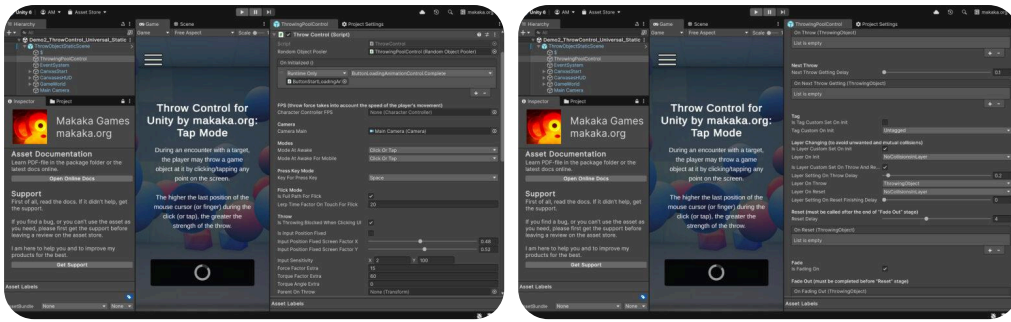
1. Random Object Pooler

RandomObjectPooler.cs — independent script that implements **Object Pool pattern**, a performance optimization technique through Objects Reusing. It's used by *ThrowControl.cs*. All Throwing Objects are created at the Scene Start from Prefabs and used throughout the Scene Session.



2. Throw Control

ThrowControl.cs — main script that manages the Throw & all Throwing Objects on a scene through **Object Pool**. The script uses disabling and enabling the following nodes: Colliders, Triggers, Renderers.



Throwing Modes

You can Assign the Throwing Modes for Desktop and Mobiles separately for the same scene.

During an encounter with a target, a player may throw any object you want by:

- ★ clicking/tapping any point on the screen;
- ★ flicking/swiping the object from the bottom of the screen up toward the target;
- ★ pressing a target key on the keyboard.

All modes form different speed and throw direction based on the last position of the mouse cursor (or finger), what makes the game even more interesting. Throw Diligently & Hit the Targets!

Throwing Force

The higher the last position of the mouse cursor (or finger) during the click (or tap, or key press), the greater the **Throwing Force** that is formed from **User Input** and parameters in the **Throwing System**:

- ★ *ThrowControl.cs* (for All Throwing Objects at once): **Input Sensitivity, Force Factor Extra.**
- ★ *ThrowingObject.cs* (for Individual Throwing Object): **Force Factor, Force Direction Extra.**

Mode 1: Click Or Tap

In this mode, the Game Object is thrown in the direction of the click (desktop) or tap (mobile).

FPS (First-Person Shooter)

For the First-Person Shooter (FPS) game, there is an option for fixing the input position in a specific place on the screen (by default in the center of the screen). So it will not depend on the mouse (or finger).

FPS Desktop Demo uses [Character Controller](#):

[Throwing Force](#) considers the speed of the player's movement.

Mode 2: Flick

In this mode, you can throw object in the style of the Pokemon GO game ([iOS](#), [Android](#)) (by flicking it from the bottom of the screen up toward the target).

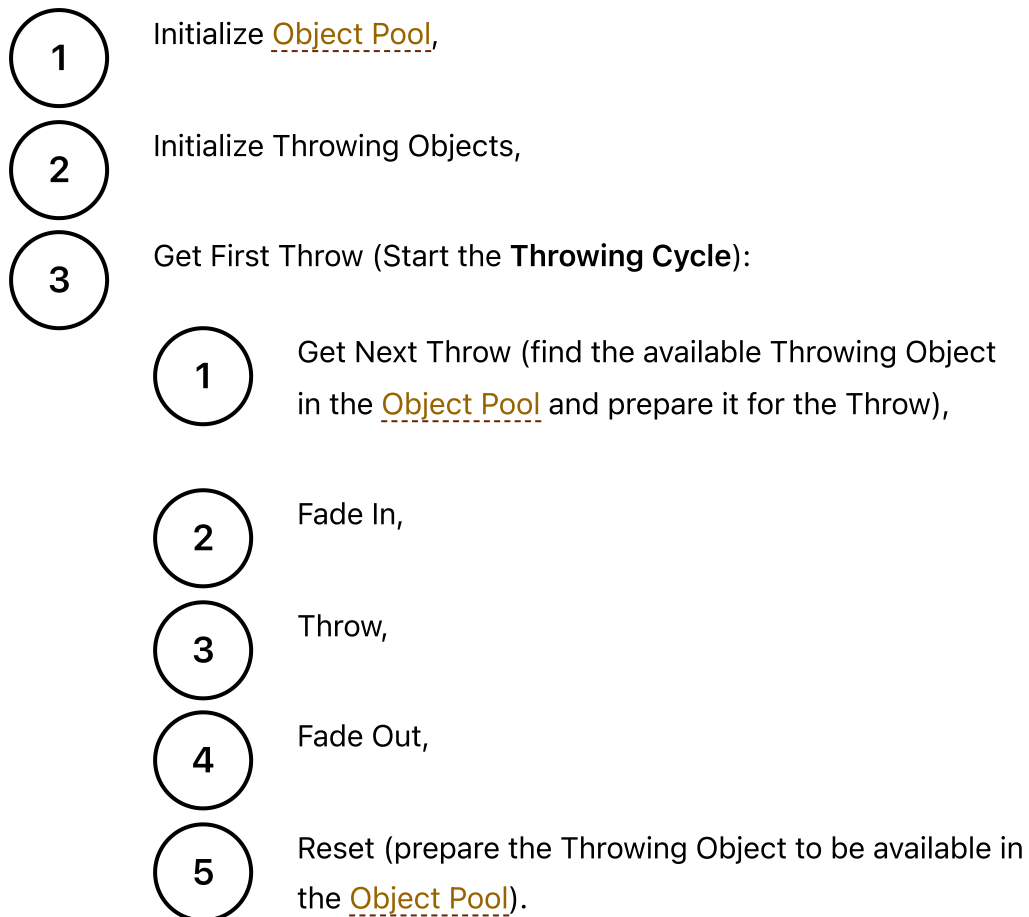
Is Full Path For Flick?

If it's false, then it allows fast flicks only: positions in the last and previous frames are taken into account.

Mode 3: Press Key

This mode is similar to [Click Or Tap mode](#), but the Throw happens after pressing the Target Key on the Keyboard. Tested with Desktop Platforms only.

Work Scheme (Stages) and Throwing Cycle



Events

- ★ OnInitialized;
- ★ Next Events have `ThrowingObject` as a parameter. So you have more flexibility with access to instances of ThrowingObject.cs in outer functions assigned to Events.
 - ★ OnNextThrowGetting,
 - ★ OnThrow,
 - ★ OnFadingOut,
 - ★ OnReset.

Coroutines

The only script that starts coroutines. Because of coroutines behavior, it's impossible to place some methods inside ThrowingObject.cs. And OOP doesn't work in this case, but we have a stable throwing with Object Pool.

Parameters

- ★ **Random Object Pooler:** Random Object Pooler component.
- ★ **On Initialized ():** event that is dispatched after All Throwing Objects have been Created and Configured — before Getting the First Throw.
- ★ FPS (throw force takes into account the speed of the player's movement):
 - ★ **Character Controller FPS:** Character Controller component is useful when creating FPS Games.
- ★ Camera:
 - ★ **Camera Main:** Camera component is intended for First-Person View and used to position Throwing Objects on the Screen.
- ★ Modes:
 - ★ **Mode At Awake:** Throwing Mode by Default.
 - ★ **Mode At Awake For Mobile:** Throwing Mode for Mobile Platforms.
- ★ Press Key Mode:
 - ★ **Key For Press Key:** Selected Key will be used when setting Throwing Mode to *Press Key*.
- ★ Flick Mode:
 - ★ **Is Full Path For Flick:** the flag will be used when setting Throwing Mode to *Flick*. If it's false, then it allows fast flicks only. Positions of the mouse cursor (or finger) in the last & previous frames are considered. False Example: **Force Factor Extra = 45** and **Input Sensitivity X = 7**.
 - ★ **Lerp Time Factor On Touch For Flick:** the value will be used when setting Throwing Mode to *Flick*. The higher the value, the faster the Throwing Object follows the mouse cursor (or finger) when held (touched).

★ Throw:

★ **Is Throwing Blocked When Clicking UI:** if it's true, Throwing will not work when clicking UI, e.g. Menu Button. It checks whether the Pointer (mouse cursor or finger) is over a UI object or not.

.

★ **Is Input Position Fixed:** it's relevant when using **Character Controller FPS**.

If it's false, then **Input Position Fixed Screen Factor X/Y** are used instead of the position of the mouse cursor (or finger).

★ **Input Position Fixed Screen Factor X, Input Position Fixed Screen Factor Y:** it's relevant when using flag: **Is Input Position Fixed**.

.

★ **Input Sensitivity:** it changes **User Input** and affects Throwing Force for All Throwing Objects at once.

★ **Force Factor Extra, Torque Factor Extra, Torque Angle Extra:** allow changing parameters described in ThrowingObject.cs but for All Throwing Objects at once with an Extra value.

★ **Parent On Throw:** it's used to avoid affecting the Movement of the Throwing Object by Movement of **Pool Parent**, described in RandomObjectPooler.cs (e.g., Main Camera). After the End of Throw during the "Reset" stage, the Throwing Object's parent will be set to the **Pool Parent**.

★ **On Throw (Throwing Object):** event that is dispatched at the Moment of the Throw: after completing all the preparations for the Throw.

★ Next Throw:

★ **Next Throw Getting Delay:** seconds to start Getting Next Throw after the Moment of the Throw.

★ **On Next Throw Getting (Throwing Object):** event that is dispatched at the first stage of the Throwing Cycle: after finding the available Throwing Object in the Object Pool and preparing it for the Throw.

★ Tag:

★ **Is Tag Custom Set On Init:** enables assigning tags for Throwing Objects at Runtime, keeping your prefabs untouched.

★ **Tag Custom On Init:** it's relevant when using flag: **Is Tag Custom Set On Init**.

★ Layer Changing (to avoid unwanted and mutual collisions):

★ **Is Layer Custom Set On Init:** allows avoiding unwanted and mutual collisions related to Throwing Objects on "Initialize Throwing Objects" stage. Layer that called *NoCollisionsInLayer* is set by Default, that is customized in the "Getting Started" Tutorial.

★ **Layer On Init:** it's relevant when using flag: **Is Layer Custom Set On Init**.

.

★ **Is Layer Custom Set On Throw And Reset:** after the Moment of the Throw, it enables interacting of Throwing Objects with the Environment and between each other.

On the "Reset" stage, it allows setting a neutral layer without collisions (*NoCollisionsInLayer* is set by Default, that is customized in the "Getting Started" Tutorial) that avoids overlapping of Throwing Objects on each other and on other physical objects.

★ **Layer Setting On Throw Delay:** seconds after the Moment of the Throw to avoid overlapping of Throwing Objects on each other and on other physical objects.

★ **Layer On Throw:** it's relevant when using flag: **Is Layer Custom Set On Throw And Reset**.

★ **Layer On Reset:** it's relevant when using flag: **Is Layer Custom Set On Throw And Reset**.

★ **Layer Setting On Reset Finishing Delay:** may be useful in very rare cases. E.g., when you play with Time Scale and Layers can't be changed quickly.

- ★ Reset (must be called after the end of "Fade Out" stage):
 - ★ **Reset Delay:** seconds to start "Reset" stage after the Moment of the Throw.
 - ★ **On Reset (Throwing Object):** event that is dispatched at the last stage of the Throwing Cycle: after preparing the Throwing Object to be available in the Object Pool.
- ★ Fade:
 - ★ **Is Fading On:** enables Fading In & Fading Out for All Throwing Objects at once. Customization for Individual prefab of Throwing Object can be done on the Material Control component.
- ★ Fade Out (must be completed before "Reset" stage):
 - ★ **On Fading Out (Throwing Object):** event that is dispatched when Fading Out actually starts — timing is synchronized with the **Delay Out** parameter of the Material Control component on Individual prefab of Throwing Object.

Public Functions

- ★ To Get the Count of Throwing Objects, use `GetObjectCount` function.
-
- ★ To Set a target prefab/prefabs of Throwing Objects before Initialization outside **Throwing System**, use `SetPrefabBeforeInit` or `SetPrefabsBeforeInit` functions.

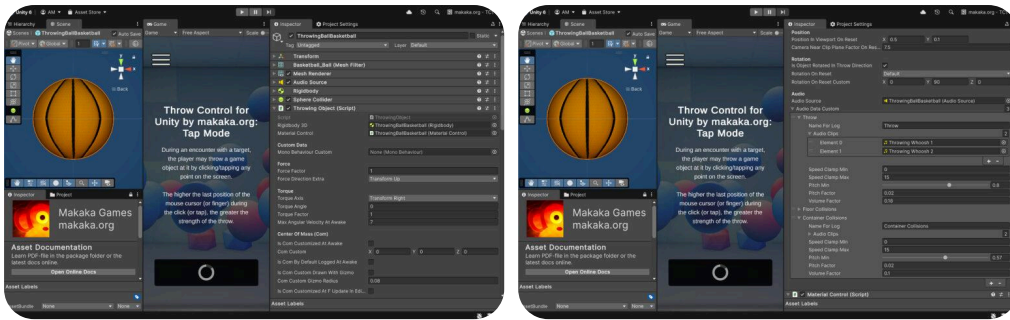
It's used in Basketball (Unity Asset) due to Different Ball Prefabs based on Game Mode (AR and Non-AR) that is chosen before Game Start — that's why you can't set it in RandomObjectPooler.cs where there is no dynamic choice.

So ThrowingPoolControl Game Object must be activated only after using this function.

3. Throwing Object

ThrowingObject.cs — script that contains basic functions and parameters to operate Throwing Object.

This script must be attached to each Throwing Prefab (check tutorial).



Events

- ★ OnThrow ,
- ★ OnResetPhysicsBase .

Parameters

- ★ **Rigidbody 3D:** [Rigidbody](#) component.
- ★ **Material Control:** [Material Control](#) component.
- ★ **Custom Data:**
 - ★ **Mono Behaviour Custom:** it's useful to assign a specific control script for a unique type of Throwing Object and access to it outside the **Throwing System**.
Example: *BasketballBallControl.cs* in [Basketball \(docs\)](#); [Football \(docs\)](#).
- ★ **Force:**
 - ★ **Force Factor:** controls the [Throwing Force](#).
 - ★ **Force Direction Extra:** Force Direction is formed automatically based on **User Input** and **Input Sensitivity**, but you can clarify the direction with an Extra value based on Axis of the Camera's Transform.
- ★ **Torque:**
 - ★ **Torque Axis, Torque Angle, Torque Factor:** all parameters form the resulting vector for [Rigidbody.AddTorque](#).
 - ★ **Max Angular Velocity At Awake:** controls [Rigidbody.maxAngularVelocity](#). It clamps Torque.
- ★ **[Center of Mass \(Com\)](#):**
 - ★ **Is Com Customized At Awake:** overrides the same [exposed property](#) of Rigidbody component just for convenience.
 - ★ **Com Custom:** it's relevant when using flag: **Is Com Customized At Awake**.
.
 - ★ **Is Com By Default Logged At Awake:** if the Center of Mass by Default is not correct, you can use it as a base point for improving with Custom value.
.
 - ★ **Is Com Custom Drawn With Gizmo:** the way for Debugging Center of Mass visually.
 - ★ **Com Custom Gizmo Radius:**
it's relevant when using flag: **Is Com Custom Drawn With Gizmo**.
.
 - ★ **Is Com Customized At F Update In Editor:** to Customize Center Of Mass without Restart in `FixedUpdate` function.

★ Position:

★ **Position In Viewport On Reset:** spawn position [x, y]. Middle is at the bottom of the screen: (0.5f, 0.1f). Y must be less than **Input Position Fixed Screen Factor Y** of [ThrowControl.cs](#) in case of using.

★ **Camera Near Clip Plane Factor On Reset:** spawn position [z]. Used for Z coordinate of 3D Position On Reset.

★ Rotation:

★ **Is Object Rotated In Thrown Direction:** affects the Rotation at the Moment of the Throw.

★ **Rotation On Reset:** affects the Rotation before the Throw.

★ **Rotation On Reset Custom:** this Vector3 will be used when setting **Rotation On Reset** to *Custom*.

★ Audio:

★ **Audio Sources:** [Audio Source](#) components. The more Audio Sources you have, the less distortion there will be (e.g., for many collisions at a time).

★ **Audio Data Custom:** array for customizing [Dynamic Sound System](#). Access from [ThrowControl.cs](#) or [ThrowingObject.cs](#) with `PlayAudioRandomlyDependingOnSpeed` function:

★ 0th Element is used for Throw Audio (Whoosh).

★ Other Elements can be used for any Custom Game Logic.

Dynamic Sound System

Dynamic Sound System allows playing Sounds based on speed, pitch, and volume factors of Throwing Objects upon:

★ Throw,

★ Custom Game Logic (e.g., for Customizing Collisions with Different Surfaces (floor, wall, etc.)). Examples of using outside the **Throwing System** dynamically and uniformly:

[Basketball \(docs\)](#), [Football \(docs\)](#).

Throw Object 3D in Unity with Dynamic Sound System...



Colliders

Center of Mass & Rotation (Torque)

There are some nuances with [Rigidbody.centerOfMass](#) when rotating.

Symmetric Mesh

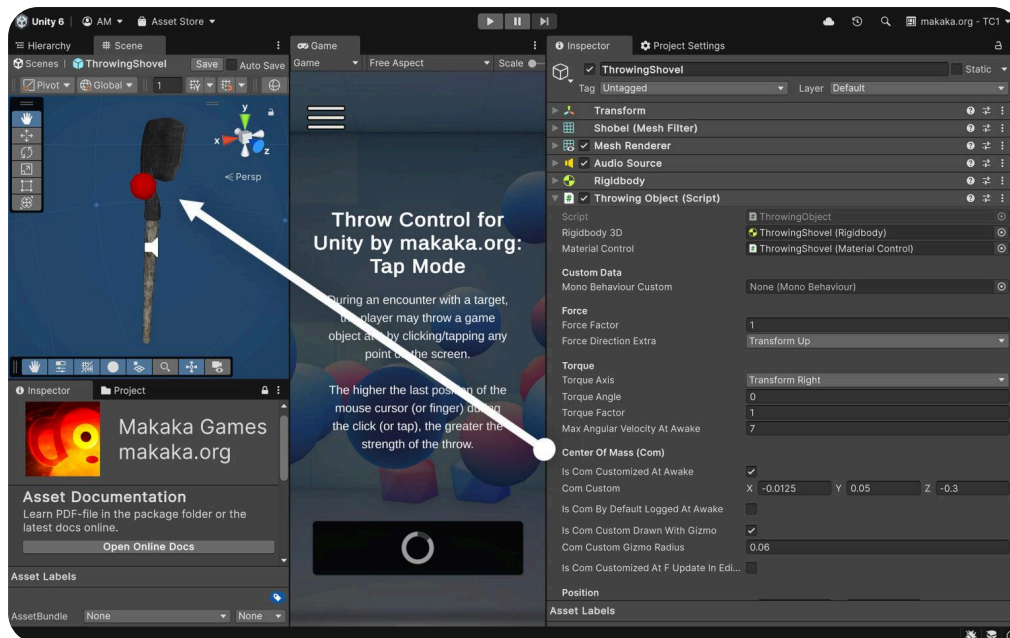
If you use Mesh Collider for Throwing Game Object and your mesh is ideally symmetric, then it's the perfect case and the game object will throw naturally.

Asymmetric Mesh

If you use Mesh Collider for Throwing Game Object and your mesh is asymmetric, then you will have unnatural throw behavior when rotating.

Solutions:

1. Custom Center of Mass. You can indicate Custom Center of Mass manually in the Editor. [Unity Package](#) contains some examples with Custom Center of Mass and Tools for Debugging it Visually (you can also turn on the Gizmos in [the Game view](#) to facilitate the debugging). There is no magic function that calculates the right Center of Mass.



2. Simple Forms of Colliders. You need to use one or more simple forms of colliders (box, sphere, etc.) placing them on the same Throwing Game Object and along the straight line. So in this case, you will have the right center of mass & right rotation.

Convex Mesh

Unity allows you to use only convex mesh colliders. In some cases, it is not what you need because the collider by default can cover more space than the object itself in some points. In order to quickly & automatically get a convex mesh collider for complex objects, use [Technie Collider Creator](#).

4. Material Control — Dynamic Material System

MaterialControl.cs — an independent script that implements **Fading (Dissolving/Appearing) VFX** for Throwing Objects as well as Material Changing in Runtime for Any Custom Game Logic (e.g., **fail**, **win**), preventing redundant memory allocations: it's used by [ThrowingObject.cs](#).

Dynamic Material System ★ Unity Asset with Fade ...

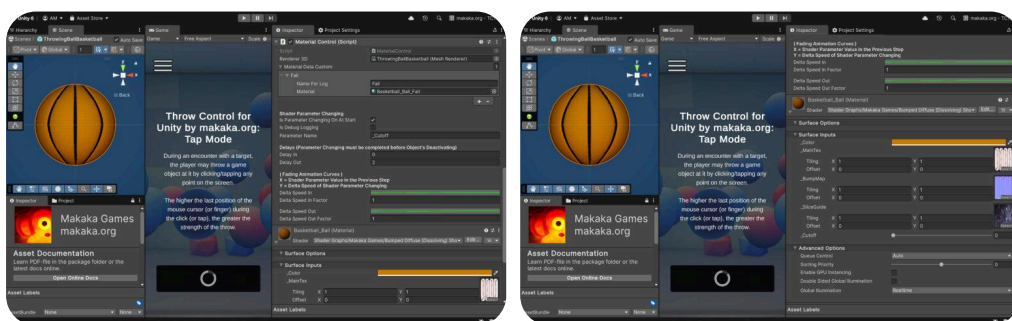


You can use this script for **Fading** (or more accurately: **Shader Parameter Changing**) outside the **Throwing System**. Examples:

- ★ Ring, Ring Holder, Net, Backboard, Pole in [Basketball Game \(docs\)](#).
- ★ Target Ring, Ring Net, Goal Net, Goal Posts in [Football \(docs\)](#).

Here you can control Fading In & Fading Out with Full Control of Min & Max Values, Delay & Speed for Indicated Shader Parameter. Actually, you can change any Float Parameter in your Shader.

The Default Material for Throwing Object is still placed in the [Mesh Renderer](#) component.



Parameters

- ★ **Renderer 3D:** [Mesh Renderer](#) component.
- ★ **Material Data Custom:** array for Customizing Material Changing for Any Custom Game Logic (e.g., [fail](#), [win](#)). Access from [ThrowControl.cs](#) or [ThrowingObject.cs](#) with `SetMaterial` function.

`SetMaterial` function is used instead of Color Changing with [Material.SetColor](#) function to avoid memory allocations. So if you plan to change the Colors of Throwing Objects in the Runtime outside the **Throwing System**, you need to create separate Materials with Target Colors in advance.

Examples with using outside the **Throwing System** dynamically and uniformly:

[Basketball \(docs\)](#), [Football \(docs\)](#).

.

- ★ **Shader Parameter Changing:**
 - ★ **Is Parameter Changing On At Start:** enables initialization of script. Allows to disable Parameter Changing for Individual prefab of Throwing Object and not for All Throwing Objects at once for which there is a global **Is Fading On** flag in [ThrowControl.cs](#).
 - ★ **Is Debug Logging:** check to understand Fading Steps. At the same time, Error Reports and Troubleshooting Tips in the Console are thrown independently of this flag.
 - ★ **Parameter Name:** indicates the Name of any Float Parameter in the Shader for changing.
- ★ **Delays** (Parameter Changing must be completed before Object's Deactivating in [Object Pool](#), i.e. before **Reset** in [ThrowControl.cs](#) — you will be notified in the Console in case of issues):
 - ★ **Delay In, Delay Out:** seconds to Start Fading In/Out after calling `Fade` function in [ThrowControl.cs](#).

- ★ Fading Animation Curves: FPS-agnostic curves that connected to Physics Simulation.

X = Shader Parameter Value in the Previous Step,

Y = Delta Speed of Shader Parameter Changing:

- ★ **Delta Speed In, Delta Speed Out**: Edge Values (Min X & Max X) display the Start and End Values for Shader Parameter when Fading. If you see Unexpected Behavior (e.g., Blinking), then the Min X value must be Synced with Parameter Start Value in the Material itself.

- ★ **Delta Speed In Factor, Delta Speed Out Factor**: for convenient Scaling of the Curve along the Y axis without changing the Curve itself.

Shaders

The Nature of the Fade depends on the `_SliceGuide` Texture on one side, and on the UV map on the other side. There are 2 Custom Shader Graphs with `_SliceGuide` Texture for Fading of Game Object and its Shadow. Check *[Your Material] > Shader > Shader Graphs > Makaka Games*:

- ★ *DiffuseDissolvingShaderGraph,*
- ★ *BumpedDiffuseDissolvingShaderGraph.*

Tutorial



*This tutorial is relevant for Throw Control 7.2.
Tutorial for the previous version can be found
only in the asset folder.*

Getting Started with Throw Control

Folders & Files in the package by default:

- ★ Makaka Games.

Steps



If you have any issues with the first launch then just Reach Support and Get Help.

- 1 Create a New Unity Project with [Unity 6000.3.11](#) & the "Universal 3D" Template.
- 2 File > Build Profiles > Platforms > Windows, Mac, Android, iOS, Web > Switch Platform.
- 3 Download and Import [Throw Control](#) into Unity.
 - 1 Go through the Warning Windows:
 - 1 Click "Import" to overwrite the Project Settings with predefined ones.
 - 2 Click "Install/Upgrade" for Unity Package Manager dependencies.
 - 2 "Import Unity Package" window:
 - 1 Click "Next".
 - 2 Click "Import".
- 4 Next Packages are provided with [Unity Package Manager](#), and they are already installed for this Asset by default. If packages are missing (Warning Window did not appear) then install them again with Package Manager:
 - 1 [uGUI 2.0.0](#):
★ **Always Required:** Window > TextMeshPro > Import TMP Essential Resources.
 - 2 [Input System 1.19.0](#).
- 5 Reopen Unity Project.
- 6 Open Scene: Makaka Games > ThrowControlX > Scenes > *Menu_ThrowControl*.

7

Option: If you use plugins for issue finding in your project like Maintainer, add this path for ignoring: *Assets/Makaka Games/ThrowControlX/PhysicMaterials/MeshesForColliders* (they were created with Technie Collider Creator which is not needed for Throw Control, but it has source files for editing).

8

Test in the Unity Editor or Build.

Notable Changes in the Default Settings

These settings are already applied by default when you import the asset. I'm posting them here just for reference when you want to use the asset in your existing project:

1

Edit > Project Settings:

- ★ Audio > DSP Buffer Size > Best Latency.
- ★ Player > Other Settings > Rendering > Color Space > Gamma (Better Colors for Demo Scenes).
- .
- ★ ***Option for macOS standalone:*** Quality:
 - ★ Select Target Level ("PC" Level is set by default for macOS): VSync Count → enable any option to avoid Screen Tearing.
 - .
- ★ Tags and Layers > Tags > + > *ThrowingObject* (used in Throw and Score Scene).

2

Customize Layer Changing (to avoid unwanted and mutual collisions):

1

Edit > Project Settings > Tags and Layers > Layers
(Use Recommended Names and Numbers, or read the instructions [below in this list](#)):

★ User Layer 8: *ThrowingObject*,

★ User Layer 9: *NoCollisionsInLayer*.

2

1. Edit > Project Settings > Physics > Settings > Shared > Layer Collision Matrix:

2. Edit > Project Settings > Physics 2D > Layer Collision Matrix:

★ *ThrowingObject*: customize as you want, or use your any existing layer instead.

★ *NoCollisionsInLayer*: uncheck all.

3

Reopen Unity Project.

4

Option: If you can't customize layers with recommended numbers, then use other empty numbers and assign them in each demo scene in [the Hierarchy window](#):

[ThrowingPoolControl](#) Game Object: [ThrowControl.cs](#)
> *Layer Changing*:

★ Layer On Init > *NoCollisionsInLayer*,

★ Layer On Throw > *ThrowingObject*,

★ Layer On Reset > *NoCollisionsInLayer*.

"\$" GameObject

Some Scenes have \$ Game Object, which has the main control script for the scene.

Various modules interact with each other with the \$.

Creating your own Throwing Object Prefab

- 1 Create Prefab with a template:
 - 1 Drag one of the customized Throwing Object prefabs from the Project window into the Scene view,
 - 2 Name your Game Object,
 - 3 Drag the named Game Object into the Project window to create the **Original Prefab**.
- 2 Select the Game Object on the Scene view & Customize it:
 - 1 Indicate Mesh of your 3D Model in Mesh Filter component,
 - 2 Customize Material in Mesh Renderer component (duplicate the previous Material in the Project window and indicate here to separate different Throwing Objects),
 - 3 Customize Fading in Material Control component,
 - 4 Customize Colliders:
 - 1 Indicate Mesh in Mesh Collider component (or customize a more suitable collider component);
 - 5 **Option:** Customize the Center of Mass for more natural Throwing.
- 3 Customize the Transform component as you wish.
- 4 Apply Changes to Prefab.
- 5 Delete Game Object from Scene.

5

Option: Add Asset Label (*ThrowingObject*) for Prefab for convenient searching in the Editor.

7

Add Prefab to the Random Object Pooler component of the ThrowingPoolControl Game Object.

8

Throw Object Forward in "Play Mode".

How to Keep Game Objects in Scene after the Throwing?

Here is the result video after applying manipulations in this section.

Throw 100 Game Objects without Fading ★ Unity A...



By default, Throwing Objects are not destroyed during the game due to Object Pool pattern with Object Reusing. Initially, the system is designed so that objects disappear after a certain time to save your device's memory. Note that objects in the pool are instantiated in advance, before you can click the start button.

You can achieve the desired result by setting numerous objects in the pool and a large reset delay while disabling the fading. To protect you from too large values, long loading at start & low performance of your game, you can't set the next values in the inspector, but you can fix the restrictions inside the script in field attributes:

1

Select Target Prefab of Throwing Object in [the Project window](#) and Disable the Fading locally:

[Material Control](#) component > **Is Parameter Changing On At Start** > Uncheck.

2

Select the [ThrowingPoolControl](#) Game Object in [the Hierarchy window](#):

1

[Random Object Pooler](#) component > **Init Pooled Amount** (set to target number of objects in pool, e.g. 99);

2

[Throw Control](#) component:

★ **Reset Delay** (set to target number of seconds: 999).

★ **Option:** You can also use the global **Is Fading On** flag instead of the local one from the [Material Control](#) component.

Throw and Score (Scene)

Cross-Platform [Throw and Score Scene](#) (Game Template) with [Sensor Camera](#) (included).

Unity Throw and Score in 3D (Game Template) ★ Th...

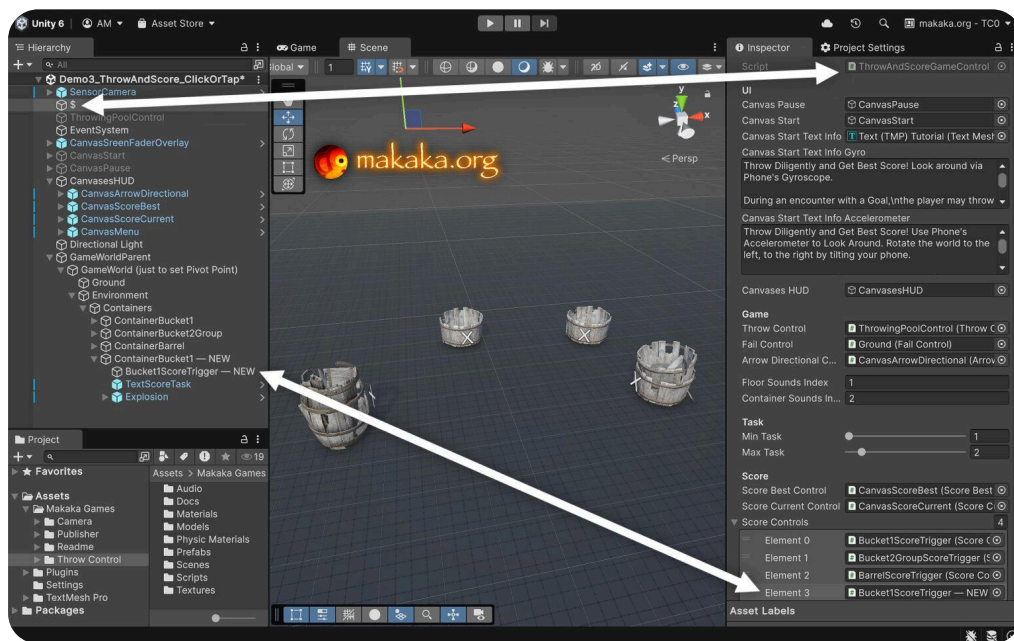


Toss Diligently and Try to Get the Best Score by Hitting the Target Containers without touching the Ground (Mode: **"Click Or Tap"**):

- ★ The Scale of the Entire Demo is smaller than in other Demos, so Physics works more quickly.
- ★ 3 Unique Containers: Barrel, Bucket & Bucket Group.
- ★ Directional Arrow that points to the Current Target.
- ★ Easy Control of the Count of Containers: place as many as you need.
- ★ Custom Meshes of Containers & their Score Triggers were created with Technie Collider Creator (it's not needed to run the project). They allow computing collisions more accurately for better playing dynamic sounds and counting points.
- ★ UI Animations and Sounds: Best Score, Container Task and Scoring, Fail.
- ★ Visual Explosions from Containers with Sound when Scoring.
- ★ Using of Dynamic Sound System & Dynamic Material System outside the **Throwing System**:
 - ★ Customized Collision Sounds of Floor and Containers using `audioDataCustom` of ThrowingObject.cs.
 - ★ Material Changing in Runtime for Custom Game Logic (**fail** – when Throwing Object touches the Ground) using `materialDataCustom` of MaterialControl.cs.
 - ★ *SameGameObjectDetector.cs* allows you to Add More Realism to the Game and Prevents Registering of Double Collisions when Scoring, Failing and Touching the Environment.
- ★ Game Randomization with **Min Task & Max Task** parameters of **Throw And Score Game Control** Component on the \$ Game Object: control how many balls should be thrown into the Next Container.

How to add a new Container?

- 1 Duplicate any Container in the Hierarchy window and position it as you want.
- 2 Select the \$ Game Object in the Hierarchy window:
 - ★ **Throw And Score Game Control Component > Score Controls:**
Add to Array the **Score Control** Component of the child Game Object of the Duplicated Container that called ... *ScoreTrigger*.
- 3 Play.



Testing

You can test Throw Control in the Unity Editor using Mouse and Keyboard.

WebGL

Learn the Article called WebGL and Unity about Building and Testing Unity games and apps for WebGL.



Throw Control asset on Mobile WebGL.

Tested with Platforms

1

Mobile Platforms:

- ★ iOS on iPhone 15;
- ★ Android on Samsung Galaxy A71;
- ★ WebGL in Google Chrome on:
 - ★ Own Website using this tutorial for iOS & Android.
 - ★ CrazyGames Website for Android.

2

Desktop Platforms:

- ★ Windows;
- ★ macOS;
- ★ WebGL in Google Chrome on:
 - ★ Own Website using this tutorial.
 - ★ CrazyGames Website.

Support

First, read the latest docs online.

If it didn't help, get the support:

- ★ Unity Discussions.
- ★ Email.

Changelog

Check the current version of [Throw Control on the Asset Store](#).

The latest versions will be added as soon as possible.

7.2 (Unity 6.3):

Improvements:

★ [Unity 6000.3.11](#) with Modern Standards of Project Settings (based on New Unity Project). The asset is downloaded into your project as a Template (Complete Project). [Read the "Getting Started" Tutorial](#) to Import it correctly.

★ [Sensor Camera 3.2](#).

★ Namespaces for All Scripts.

★ Android: Vulkan Graphics API first.

★ Regulate SFX volumes.

7.1:

Features:

- ★ Cross-Platform **Throw and Score** Scene (Game Template) with Sensor Camera (included).
Toss Diligently and Try to Get the Best Score by Hitting the Target Containers without touching the Ground (Mode: **"Click Or Tap"**):
- ★ The Scale of the Entire Demo is smaller than in other Demos, so Physics works more quickly. The Scale of the Next Throwing Objects was specifically reduced to work there:
 - ★ Axe, Basketball (Ball), Sword, Broken Sword.
- ★ 3 Unique Containers: Barrel, Bucket & Bucket Group.
- ★ Directional Arrow that points to the Current Target.
- ★ Easy Control of the Count of Containers: place as many as you need.
- ★ Custom Meshes of Containers & their Score Triggers were created with Technie Collider Creator (it's not needed to run the project). They allow computing collisions more accurately for better playing dynamic sounds and counting points.
- ★ UI Animations and Sounds: Best Score, Container Task and Scoring, Fail.
- ★ Visual Explosions from Containers with Sound when Scoring.
- ★ Using of Dynamic Sound System & Dynamic Material System outside the **Throwing System**:
 - ★ Customized Collision Sounds of Floor and Containers using `audioDataCustom` of ThrowingObject.cs.
 - ★ Material Changing in Runtime for Custom Game Logic (**fail** – when Throwing Object touches the Ground) using `materialDataCustom` of MaterialControl.cs.
 - ★ SameGameObjectDetector.cs allows you to Add More Realism to the Game and Prevents Registering of Double Collisions when Scoring, Failing and Touching the Environment.
- ★ Game Randomization with **Min Task** and **Max Task**: control how many balls should be thrown into the Next Container.
- ★ ThrowingObject.cs now uses **Audio Sources** array instead of the only **Audio Source**. The more Audio Sources you have, the less distortion there will be (e.g., for many collisions at a time).

Improvements:

★ Unity 6000.0.44.

7.0:

Improvements:

★ Unity 6000.0.38.

★ URP (New Standard in Unity starting with Unity 6): *instead of BRP.*

★ Input System (New Standard in Unity) for All Throw-related Scripts: *instead of Input Manager.*

★ Supplement Documentation for Professional Use and Add Tooltips in Scripts.

★ **Material Control:**

★ ***Optional Flag in Editor: Is Debug Logging*** — to understand Fading Steps.

★ **Error Reports and Troubleshooting Tips** in the Console (thrown independently of the ***flag***).

★ Merge *parameterAtStart*, *parameterMin*, *parameterMax* variables with values on the X-axis in Fading Animation Curves:

★ X = Shader Parameter in the Previous Step,

★ Y = Delta Speed of Shader Parameter Changing.

★ Enable **Fading In** and **Fading Out** for All Game Objects on "Flick" Scene. Game Objects on Other Scenes provide **Fading Out** only by default.

★ Shorten and Rename the Folder of the Asset and All Scenes to fit the Asset Name.

Changes:

★ **Deprecated:**

★ Particles Cutout Transparent (shader).

★ **Flag Custom** field in ThrowingObject.cs.

Any Custom Data must be placed in the **Mono Behaviour Custom**.

Fixes:

- ★ First Appearance of Game Object using Fading In.
- ★ Apply Correction for Mouse Sensitivity on Desktop FPS Demo for WebGL platform.

6.0:

Features:

- ★ New Throwing Mode for Desktop using Keyboard called "Press Key" (in addition to Mouse only) with 2 Ways that form speed and throw direction based on:
 - ★ current input position of Mouse (last position of mouse cursor),
 - ★ fixed input position (doesn't depend on the mouse). It's suitable for FPS.
- ★ WebGL Support via HTTPS for Mobiles: iOS & Android. [Learn WebGL Tutorial.](#)
- ★ New Throwing Object: [Football ball.](#)

Improvements:

- ★ [Unity 2022.3.20.](#)
 - ★ [Watch some part of the DevLog](#) during upgrading from Unity 2021.3.
- ★ Asset is downloaded into your project as a Tool/Plugin now (not as a Complete Project), which is more flexible. Read the ["Getting Started" Tutorial](#) to Import correctly.
- ★ Assigning Throwing Modes for Desktop and Mobiles separately.
- ★ Detailed Tutorial Screens for 4 Demo Scenes.

5.2:

Features:

★ Opportunity to Set a target prefab/prefabs of Throwing Objects before Initialization outside **Throwing System** with the next functions of [ThrowControl.cs](#) (it's used in [AR Basketball \(docs\)](#) due to different ball prefabs for different AR Engines: AR Foundation & [AR Camera Lite \(docs\)](#)):

★ `public void SetPrefabBeforeInit(GameObject gameObject) ,`

★ `public void SetPrefabsBeforeInit(GameObject[] gameObjects) .`

Improvements:

★ [Unity 2021.3.4.](#)

5.1:

Improvements:

★ [Unity 2021.2.16.](#)

★ Opportunity to use a Target Audio Source when Playing Sounds from `audioDataCustom` related to Throwing Object with `public void PlayAudioRandomlyDependingOnSpeed(int index, bool isStoppedBeforePlay, AudioSource audioSource = null)` & [ThrowingObject.cs](#).

Example: [Basketball \(docs\)](#), [Football \(docs\)](#).

5.0:

Features:

- ★ Dynamic and Unified Operating of Data for Different Throwing Objects (examples with using outside the **Throwing System**: [Basketball \(docs\)](#), [Football \(docs\)](#)):
 - ★ `audioDataCustom` – Array for Customizing Collisions with different surfaces (e.g., floor, wall) or for Any Custom Game Logic; merged with deprecated `whooshAudio`. Access from [ThrowControl.cs](#) or [ThrowingObject.cs](#) with `PlayAudioRandomlyDependingOnSpeed` function.
 - ★ `materialDataCustom` ([MaterialControl.cs](#)) – Array for Material Changing for Any Custom Game Logic (e.g., **fail**, **win**). Access from [ThrowControl.cs](#) or [ThrowingObject.cs](#) with `SetMaterial` function.
- ★ [RandomObjectPooler.cs](#): Split `areRandomizedObjects` boolean flag into 2 flags:
 - ★ `areRandomizedObjectsWhenCreating` – when you need a random instance count of possible Throwing Object prefabs every Scene Start.
 - ★ `areRandomizedObjectsWhenGetting` – when you require a random Throwing Object instance for every Throw.
- ★ Retrieving Count of Throwing Objects with [ThrowControl.cs](#) > `GetObjectCount` function.
- ★ Tools for Debugging the Center of Mass Visually in Unity Editor. If the Center of Mass by Default is not correct, you can use these tools for improving Center of Mass with Custom value.

Improvements:

★ Unity 2021.2.13.

★ Instantiating Throwing Objects at Start outside the Game Area in target positionAtInit ([RandomObjectPooler.cs](#)) — Avoid Collisions between Throwing Objects and with Other Physical Objects more accurately:

★ Split Reset of Position:

★ On Init – to Target Position at Instantiating.

★ On Throw – to Camera Position.

★ Split Layer Changing option into 2 stages & 3 states:

★ On Init,

★ On Throw & On Reset.

★ Consolidating the Same Code of Registering Throwing Object in **Throwing System** into [RandomObjectPooler.cs](#).

★ [ThrowControl.cs](#) > OnInitialized Event is raised more accurately: in the correct frame after all operations.

★ Shovel Throwing Object: Custom and Accurate Collider for Shovel.

★ Unified Menu Scene for All Scenes.

★ Loading Animation for Start Buttons.

4.4:

- ★ Unity 2021.1.20;
- ★ New Flag: "Is Throwing Blocked When Clicking UI";
- ★ Improve Project Settings to modern standards of New Unity Project;
- ★ Fix Resetting Rotation & Position from Rigidbody to Transform to be compatible with AR Foundation camera (for [AR Throwing \(docs\)](#));
- ★ Fix Secondary Throw in "Flick" Mode when clicking on current Throwing Object after throw and before appearing of new throwing object;
- ★ Fix Throwing in "Flick" Mode when clicking outside any object with Collider after correct flicking throw;
- ★ Fix Throwing in "Flick" Mode when !isFullPathForFlick : clicking on empty space doesn't cause throwing now;
- ★ Fix "NullReferenceException" when "None" Game Object is assigned in Inspector of [Random Object Pooler](#) component;
- ★ Fix Missing Textures.

4.3:

- ★ New Platform: WebGL Support (Tested in Google Chrome);
- ★ Unity 2021.1.12;
- ★ Fix Dissolving VFX.

4.2:

- ★ Optimization: [TextMesh PRO](#) for all texts.

4.1:

- ★ Unity 2021.1.6.

4.0:

This version adds performance improvements by more effective handling of materials, shaders, fading & caching, so it's incompatible with previous versions of [Asset](#).

- ★ Improve Dissolving (Fading) VFX. Separate Independent Script for Throwing Objects: [MaterialControl.cs](#) — so you can use it for Fading outside **Throwing System** (Examples: Ring, Ring Holder, Net, Backboard, Pole in [AR Basketball GO \(docs\)](#) & [Basketball Game \(docs\)](#)).
- ★ 1 Material Instead of 2: No Memory Allocation while Fading.
- ★ 3 Custom Shaders with Shadow Fading & "Slice Guide" Texture for Fading:
 - ★ Diffuse,
 - ★ Bumped Diffuse,
 - ★ Particles Cutout Transparent — for Transparent Textures (Example: Net in [AR Basketball GO \(docs\)](#) & [Basketball Game \(docs\)](#)).
- ★ Fading In & Fading Out with Full Control of Value, Delay, Time & Speed for Indicated Shader Parameter.
- ★ Add public event Actions for Each Throwing Object in [ThrowingObject.cs](#):
 - ★ OnThrow;
 - ★ OnResetPhysicsBase.
- ★ Improve all public Events in *ThrowControl.cs*: add *ThrowingObject* as a parameter. So you have more flexibility with access to instances of [ThrowingObject.cs](#) in outer functions assigned to Events.
- ★ Add a **Mono Behaviour Custom** field in [ThrowingObject.cs](#): it is useful to assign a specific control script for a unique type of Throwing Object and access to it outside the Throwing System. Example: *BasketballBallControl.cs* in [AR Basketball GO \(docs\)](#) & [Basketball Game \(docs\)](#).
- ★ Now *MaterialControl.SetMaterial* function is used instead of [Material.SetColor](#) function all over the system to avoid memory allocations. So if you plan to change Colors of Throwing Objects in Runtime outside **Throwing System**, you need to create Materials with Target colors in advance. Example: Red Fail Material in [AR Basketball GO \(docs\)](#) & [Basketball Game \(docs\)](#).
- ★ Unity 2019.3:
 - ★ Fix Mesh Collider of Horseshoe.

★ `AudioData` class and `PlayAudioRandomlyDependingOnSpeed` function. Now you can easily declare Sound Settings in your own scripts to Play Sounds based on the speed of Throwing Objects. It's helpful when you implement a Dynamic Sound System when Throwing Objects collide with Floor, Walls and Other Game Objects.

★ Function: `public int GetObjectCount();`

★ Function: `public void SetColor(Color color, GameObject to) .`

3.16 (Bug Fixing):

★ Audio Delay (Latency) on Mobiles;

★ "Index Out of Bounds" error in `Input.GetTouch(0)` for Mobiles;

★ Using of `Debug.LogWarning` function for unknown platforms.

3.15:

★ Option: **Flag Custom** field in ThrowingObject.cs for Any User Logic.

3.14:

★ Option: **Tag Selector** field for ThrowControl.cs to set a custom tag for all objects in the pool.

3.13:

★ Unity 2019.1;

3.12:

★ Unity 2018.3;

3.7:

- ★ Move cameraNearClipPlaneFactor to [ThrowingObject.cs](#) to customize the whole position (X, Y, Z) of Throwing Object prefab in one place;
- ★ Fix enabling and disabling colliders for compound Throwing Prefabs.

3.6:

- ★ Unity 2018.2;
- ★ Fix errors: When Flick Mode you just click on the object without finger position change.

3.4 (Layer Changing in Throw Control):

Actual for quick Throwing to neutralize mutual collisions.

- ★ Customizations:
 - ★ ON / OFF,
 - ★ Layer Mask on Throw,
 - ★ Layer Mask on Reset,
 - ★ Delay.

3.3 (Throw Customizations in Throw Control):

- ★ Input Sensitivity,
- ★ Force Factor Extra,
- ★ Torque Factor Extra,
- ★ Torque Angle Extra,
- ★ Parent on Throw.

3.2 ("Flick" Mode Customizations in Throw Control):

- ★ Option: **Is Full Path For Flick:**
 - ★ If it's false then it allows fast flicks only (positions in the last and previous frames are taken into account);
- ★ **Lerp Time Factor On Touch.**

3.1:

- ★ FPS Demo integrated with Character Controller component (Desktop);
- ★ Throw force takes into account the speed of the player's movement;
- ★ Fixing an input position in a specific place on the screen (by default in the center of the screen).

3.0 (New Architecture: Throw Control):

Main Control Script (Throw Control) operates Object Pool & All Throwing Objects now (Throwing Object Script is attached to each Throwing Object prefab).

Work Scheme:

- 1 Initialize Pool,
- 2 Initialize Throwing Objects,
- 3 Get the First Throw,
- 4 Get Next Throw.

★ Customizations in Throw Control:

★ Throw Mode;

★ Events with Delays:

★ OnInitialized ,

★ OnThrow ,

★ OnNextThrowGetting ,

★ OnReset ,

★ OnFadeOut .

2.6 (Rotation Customizations):

★ Option: Is Object Rotated In Thrown Direction;

★ Rotation On Reset:

★ Default,

★ Last,

★ Random,

★ Custom.

2.5:

★ Position In Viewport On Reset.

2.4 (Center of Mass Customizations):

★ Custom Center of Mass,

★ Logging of Center of Mass by Default.

2.3 (Torque Customizations):

★ Torque Axis,

★ Torque Angle,

★ Torque Factor,

★ Max Angular Velocity at Awake.

2.2 (Force Customization):

★ Force Factor,

★ Force Direction Extra.

2.1:

★ "Click or Tap" Throwing Mode for Desktop & Mobiles;

★ Customization:

★ Throwing Mode.

2.0 (Object Pool — Throwing of Multiple Objects):

★ Prefabs Using:

- ★ Single (actual for Testing target prefab; None => Multiple),
- ★ Multiple;

★ Generation order for Multiple Prefabs:

- ★ In random order,
- ★ In the right order;

★ Customizations:

- ★ Pool Parent,
- ★ Init pooled amount,
- ★ Event (OnInitialized).

1.3 (Dynamic Sound System):

Sound System depending on Speed of Throwing Object.

★ Customizations:

- ★ Array for randomly selecting Sounds,
- ★ Speed Clamping,
- ★ Pitch (Minimum & Factor),
- ★ Volume Factor.

1.2 (Fade Out option for Thrown Objects):

★ Customizations:

- ★ ON / OFF,
- ★ Delay,
- ★ Speed,
- ★ Materials,
- ★ Event (OnInitialized).

1.1:

- ★ Desktop version: throw object with the mouse. Use it for Standalone apps or for testing of mobile apps in Unity Editor.

1.0 ("Flick" Throwing Mode for Mobiles):

★ Throwing of Single Object in “Flick” Mode for Mobiles ([AR Basketball GO — Unity Asset](#)).